Algorithms for Data Science
Practice Exam

See the previous exercise sheets for more examples of tasks that can appear in the exam.
Note: in the questions asking you to "explain", include a brief explanation of your answer (a few words).

Task 1:

What are the time complexities of the following programs in Θ notation in dependence of n?

(a)

```
for x = 1 to n do
    a = a + 1
    for y = x − 10 to x + 10 do
        a = a + 1
```

Time: _____

(b)

```
for x = 1 to 10n do
    y = 10n
    while y ≥ x do
        y = y − 1
```

Time: _____

Task 2:

Suppose we have an algorithm X which, given the input x, proceeds as follows:

- if x is of size at most 1, it computes the output in time $O(1)$,
- if x is of size $n > 1$, it recursively calls X on four inputs each of size $\lceil n/2 \rceil$. After the four calls, it combines the results and computes the output in time $\Theta(n^2)$.

What is the time complexity of the algorithm X in Θ notation? Explain your answer.

Task 3:

We are given a directed graph with n vertices numbered from 0 to $n-1$, and m edges given as a list of pairs (source vertex, target vertex). What algorithm can we use to sort the edges in time and memory $O(n+m)$? The edges should be sorted by their source vertices, and edges from the same vertex should be sorted by their target vertex. Explain your answer.

Task 4:

Write an algorithm based on dynamic programming that solves the same problem as `AlgoRec(`*a, n*`)`. Assume that n is an integer of size at least 2 and that a is an array $a[2..n]$ of length $n-1$ starting at index 2.

Hint: You don't have to understand the problem. You just have to transform the algorithm below.

```
memo = new dictionary
AlgoRec(a[2..n], p)
    if p < 2 then
        return 0
    if memo.Check(p) == False then
        memo.Add(p, max(a[p] + AlgoRec(a,p-2), AlgoRec(a,p-1)))
    return memo.Get(p)
```

Task 5:

Draw a binary search tree of height 3 containing the keys $40, 10, 30, 20$ such that it is *not* an AVL tree. Mark the vertex in your tree that violates the invariant of the AVL tree.

Task 6:

In each subtask, suppose that our algorithm executes the given number of dominating operations in the wort case where n is the size of the input. Give the time complexity simplified as far as possible using the Θ notation.

(a) $4n + \log n + 1$

Answer: _____

(b) $3n \log n^3 + 100n$

Answer: _____

(c) $10 \log^{10} n + \sqrt{n}/100 + \sqrt{2}^{\log n} \cdot \log n$

Answer: _____

Task 7:

Give the worst case time complexity of the following algorithms in Θ notation in dependence of n:

(a)

```
Input: an array a[1..n]
J = make_heap(a)
for p = n to 1 do
    J.removeMax()
```

Time: _____

(b)

```
Input: edges: a list with O(n^2) edges of a graph that has n vertices numbered from 1 to n
for v = 1 to n do
    for w = 1 to n do
        δ = Floyd_Warshall(edges)
        print(δ[v][w])
```

Time: _____

(c)

```
Input: an array a[1..n], a sorted array b[1..n]
for i = 1 to n do
    Add a[i] into b such that b is still sorted
```

Time: _____

(d)

```
Input: an AVL tree T with n elements, an array a[1..100n^2] with 100 · n^2 elements.
for i = 1 to 100 · n · n do
    T.add(a[i])
```

Time: _____

Task 8:

Give the worst case *extra*[1] memory complexity of the following algorithms in Θ notation in dependence of n:

(a)

Input: an array $a[1..n]$
`merge_sort(`a`)`
return $a[0]$

Memory: _____

(b)

Input: an array $a[1..n]$
$x = 2 \cdot a[0]$
return `binary_search(`a, x`)`

Memory: _____

(c)

Input: a list *adjacency_list* representing a weighed graph with n vertices numbered from 1 to n and $O(n)$ edges,
two vertices s and t (given as numbers from 1 to n)
$dist$ = `dijkstra_algorithm(`*adjacency_list*, s`)`
return $dist[s][t]$

Memory: _____

(d) The same algorithm as before, but this time the graph has $\Theta(n^2)$ edges.

Memory: _____

[1] not counting the memory for the input

Task 9:

Consider the following Python and C++ programs labeled as A, B, C, and D, operating on a list/vector a containing n integers.

Order the four programs by their actual real running time assuming that n is very large and that the programs are executed on the same computer. Briefly explain your reasoning for the ordering.

Recall that the Python operation `a.pop(i)` removes $a[i]$ from a. If you are unfamiliar with C++, it suffices for you to know that Program C does the same as Program A, and Program D does the same as Program B.

- Program A in Python

```
del a[0]
for _ in range(n // 2):
        a.pop(0)
```

- Program B in Python

```
for _ in range(n // 2):
        a.pop(len(a)-1)
```

- Program C in C++

```
a.erase(a.begin());
for (int i = 0; i < n / 2; ++i) {
        a.erase(a.begin());
}
```

- Program D in C++

```
for (int i = 0; i < n / 2; ++i) {
        a.pop_back();
}
```

Task 10:

Is it possible to sort n elements with $O(n\alpha(n))$ comparisons in the worst case (where $\alpha(n)$ is the inverse Ackermann function)? Explain.

Task 11:

What is the most appropriate data structure for the following scenario (in terms of memory and time complexity) from the list below? Give a short explanation of your answer.

In your answer, mention the memory and time complexity of your data structure. Also argue why your data structure is more appropriate than the other ones.

Initially, there are n bacteria on a petri dish, numbered from 1 to n. The bacteria are growing and whenever two bacteria meet, they become a new single bacterium. A camera registers such events and sends the data to a computer. Propose a data structure that can quickly tell whether any given two initial bacteria (given by their ID numbers) already belong to a same new bacterium or are still isolated from each other.

1. Array indexed by keys
2. AVL tree
3. Find-union
4. Hash table
5. Heap/Priority queue
6. Sorted Linked list
7. Queue
8. Stack
9. Sorted array
10. Unsorted array

Most appropriate data structure: _____

Explanation:

Task 12:

Consider the following incomplete Python function to determine whether it's possible to reach the last field from the first field given the constraints. The input is a Python list (array) *level* of length $n \geq 4$ containing only values 0 and 1, as well as a positive integer w. The goal is to reach field $n-1$ from field 0. If $level[i] = 1$, you can walk to the right, or jump to the field $i + w$. You can jump at most three times. If $level[i] = 0$, you can't move further.

Note: If you have problems with the first subtask, take a look at the second subtask to get a right idea for the algorithm.

```
1  def can_reach(level, w):
2      n = len(level)
3      # ??? [Line 3]
4      # ??? [Line 4]
5      # ??? [Line 5] (Starts with for...)
6          if level[i - 1] == 1:
7              # ??? [Line 7]
8          if i - w >= 0 and level[i - w] == 1:
9              # ??? [Line 9]
10     # ??? [Line 10] (Starts with return...)
```

(a) Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct label from the code snippets below:

- Line 3: _____
- Line 4: _____
- Line 5: _____
- Line 7: _____
- Line 9: _____
- Line 10: _____

Code snippets to use:

- (A) `return dp[n - 1] <= n`
- (B) `dp[i] = dp[i - 1]`
- (C) `dp[i] = min(dp[i], dp[i - w] + 1)`
- (D) `for i in range(1,n):`
- (E) `dp[i] = min(dp[i - 1], dp[i - w] + 1)`
- (F) `dp = [0] * n`
- (G) `for i in range(0,n):`
- (H) `dp = [n] * n`
- (I) `return dp[n - 1] > -1`
- (J) `dp[0] = 0`
- (K) `dp = [-1] * n`
- (L) `return dp[n - 1] < 4`
- (M) `dp[i] = dp[i - w] + 1`
- (N) `dp[i] = min(dp[i] + 1, dp[i - w])`
- (O) `dp[0] = n`
- (P) `dp[0] = -1`
- (Q) `for i in range(1,n-1):`
- (R) `dp[0] = 1`
- (S) `dp[i] = dp[i - 1] + 1`
- (T) `for i in range(0,n-1):`
- (U) `return dp[n - 1] >= 3`
- (V) `dp[i] = dp[i - w]`
- (W) `dp[i] = min(dp[i - 1] + 1, dp[i - w])`

(b) Choose the correct invariant for the for loop in line 5 from the options below:

A. For $j < i$: $dp[j]$ is the minimum number of jumps to reach field j, or it is n if the field is unreachable.

B. For $j < i$: $dp[j]$ is the minimum number of jumps to reach field j, or it is -1 if the field is unreachable.

C. For $j < i$: $dp[j]$ is the minimum number of moves (walks and jumps) to reach field j, or it is n if the field is unreachable.

D. For $j < i$: $dp[j]$ is the minimum number of moves (walks and jumps) to reach field j, or it is -1 if the field is unreachable.

Task 13:

Consider the following decision problem A:

Input: a graph G and two vertices s and t. Is there a path from s to t?

Consider the following decision problem B:

Input: a weighted graph G. Is there a tour going through all the n vertices (i.e., visiting each vertex at least once), such that the total edge weight of the tour is exactly n?

Mark all the correct sentences and give a short explanation.

- ◯ A can be reduced to B in polynomial time.
- ◯ B can be reduced to A in polynomial time.
- ◯ A is in P.
- ◯ B is in P.
- ◯ It is unknown whether A can be reduced to B in polynomial time.
- ◯ It is unknown whether B can be reduced to A in polynomial time.
- ◯ It is unknown whether A is in P.
- ◯ It is unknown whether B is in P.